

VSAL: A Vision Solver with Adaptive Layouts for Graph Property Detection

Jiahao Xie
jiahaox@udel.edu
University of Delaware
Newark, Delaware, USA

Guangmo Tong
amotong@udel.edu
University of Delaware
Newark, Delaware, USA

Abstract

Graph property detection aims to determine whether a graph exhibits certain structural properties, such as being Hamiltonian. Recently, learning-based approaches have shown great promise by leveraging data-driven models to detect graph properties efficiently. In particular, vision-based methods offer a visually intuitive solution by processing the visualizations of graphs. However, existing vision-based methods rely on fixed visual graph layouts, and therefore, the expressiveness of their pipeline is restricted. To overcome this limitation, we propose VSAL, a vision-based framework that incorporates an adaptive layout generator capable of dynamically producing informative graph visualizations tailored to individual instances, thereby improving graph property detection. Extensive experiments demonstrate that VSAL outperforms state-of-the-art vision-based methods on various tasks such as Hamiltonian cycle, planarity, claw-freeness, and tree detection.

CCS Concepts

• Computing methodologies → Supervised learning; • Theory of computation → Graph algorithms analysis.

Keywords

Graph Property Detection; Layout; Vision; Hamiltonian Cycle.

ACM Reference Format:

Jiahao Xie and Guangmo Tong. 2026. VSAL: A Vision Solver with Adaptive Layouts for Graph Property Detection. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792224>

Resource Availability:

The source code and data of this paper have been released at <https://doi.org/10.5281/zenodo.18332925> and <https://github.com/Jiahao-Xie-86/VSAL>.

1 Introduction

Graphs are a fundamental data structure for modeling the web, representing various structures such as hyperlink networks [4], social networks [37], and clickstream data [36]. Detecting structural properties of these web graphs, such as Hamiltonicity [18], planarity [25], and connectivity [7], is significant for a wide range of web applications, including community detection [30], network visualization [41], hyperlink prediction [8], and anomaly detection [26].

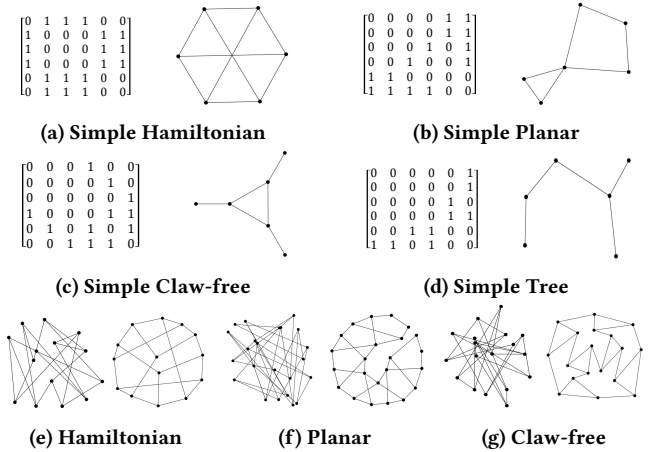


Figure 1: Each pair of (a)–(d) shows a simple graph with a certain property, including the adjacency matrix (left) and a well-designed layout generated by VSAL (right). Each pair of (e)–(g) shows a complex graph with a random layout (left) and a well-designed layout generated by VSAL (right).

However, traditional algorithmic methods for graph property detection often require special designs for individual properties and also struggle with scalability [3]. More recently, statistical learning methods have emerged as data-driven alternatives for handling such tasks, providing greater flexibility and efficiency [48].

Despite their advantages, existing learning methods primarily encode graph information through adjacency matrices, making them less effective for tasks involving spatial and visual interpretation [39], such as identifying geometric symmetries [13]. In this regard, vision-based methods offer a promising alternative [33, 34, 38]. One particular pipeline of such methods is VN-Solver [38] consisting of three conceptual steps: a) transforming graphs into 2D layouts, b) visualizing layouts as images, and c) detecting graph properties by image classifiers. Such a pipeline explores an interesting concept: can we detect graph properties by taking a look at graph visualizations? Intuitively, many graph properties can be much more evident from visualizations than from matrices; for example, the graph properties in Figs. 1a–1d can be easily confirmed from well-designed layouts, but it is arguably less straightforward for humans to do the same by scrutinizing the adjacency matrices. As for larger and more complex graphs, visual patterns may become challenging to interpret manually (e.g., Figs. 1e–1g), but recent advances in image classification provide effective tools for such purposes, such as ResNet [22] and Vision Transformer (ViT) [11]. Indeed, it has been shown that such a framework is not only feasible but also



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792224>

comparable to the state-of-the-art matrix-based methods for the Hamiltonian cycle problem [38].

Contribution. Although VN-solver has achieved promising performance, it relies on fixed layouts that are not adaptable to the underlying data distribution. In this light, we propose Vision neural Solver with Adaptive Layouts (VSAL), a new framework aiming to create layouts that can mimic principled designs while being learnable to capture instance-specific properties based on classification feedback. To realize such an idea, we leverage the idea of generative adversarial networks [17] to generate dynamic layouts, and design a differentiable visualization module to enable end-to-end training. To our delight, VSAL can learn to generate layouts that can not only visually clarify graph structural properties (Figs. 1e-1g), but also enable significantly better detection performance than VN-Solver on multiple graph property detection tasks (as shown in Sec. 5). Furthermore, VSAL functions as intended; for example, it can effectively optimize various principled graph layouts, and the generated layouts exhibit more clarity (e.g., reducing edge crossings) and can capture key features for better detection (e.g., identifying isolated and one-degree nodes for Hamiltonian cycle detection).

2 Related Work

Matrix-based learning methods. Existing statistical learning methods for graph property detection are primarily matrix-based, as summarized in recent surveys [31, 35]. Graphormer [49] adapts the Transformer [46] to graphs by incorporating three structural encodings: centrality (capturing node importance via degree measures), spatial (encoding pairwise shortest-path distances), and edge (integrating edge features into attention). These enhancements enable Graphormer to capture global dependencies and achieve state-of-the-art performance on graph property detection tasks [13, 28]. Building on this architecture, Graphormer-GD [50] introduces Gaussian kernel-based distance encoding to better model structural similarities and improve generalization ability. EquiformerV2 [32] extends the Transformer with SE(3)-equivariance, allowing it to exploit geometric symmetries and directional information, which is especially beneficial for spatially structured graphs. More recently, GraphsGPT [16] introduces a pure Transformer architecture that encodes graphs as sequences of learned Graph Words via a Graph2Seq encoder and reconstructs them through an edge-centric autoregressive decoder, yielding expressive representations for downstream tasks. While effective, these matrix-based methods are less capable of capturing visual patterns, as the adjacency matrices primarily represent relational node interactions [5].

Vision-based learning methods. There have been several attempts to integrate vision techniques into optimization tasks involving graphs. For example, Ling *et al.* [33, 34] employ CNNs to directly process raw graph visualizations without designing specific layouts, and Graikos *et al.* [19] explore diffusion models for iterative graph pattern generation. However, these methods cannot be directly applied to graph property detection tasks. VN-Solver [38] is the first vision-based method that utilizes visualized graph layouts for graph property detection. By employing ResNet-50 to process the visualizations of fixed circular and spiral graph layouts, VN-Solver tackles the Hamiltonian cycle problem and achieves encouraging performance. However, its reliance on fixed layouts

limits its expressiveness and flexibility in capturing complex graph structures. In contrast, VSAL overcomes these limitations by leveraging a Wasserstein adversarial generation process [1] to adaptively generate graph layouts under the guidance of a classifier, thereby improving detection accuracy and efficiency.

Graph layout generation strategies. Traditional graph layout algorithms, such as force-directed methods [27], visualize graphs by optimizing certain aesthetic criteria, such as minimizing edge crossings. However, these methods require manual parameter tuning and lack generalization ability across diverse graphs. Recent work has introduced generative models for graph layout generation. For instance, Kwon *et al.* [29] develop a variational autoencoder that learns layout distributions from examples, enabling smooth transitions between different layout styles. Similarly, Wang *et al.* [47] propose a generative adversarial framework that optimizes layouts for diverse aesthetic goals such as stress minimization and crossing angle maximization. While these approaches focus on producing aesthetically pleasing layouts, our work takes a different direction by generating layouts explicitly optimized to improve graph property detection accuracy. By conditioning layout generation on classification feedback, VSAL creates layouts that make structural properties more visually distinguishable for the classifier.

3 Preliminary

Graph property detection is, in essence, a binary classification task. Let $\mathcal{G}$ denote the space of all undirected graphs and $\mathcal{Y} = \{0, 1\}$ represent the set of binary labels, where $y = 1$ (resp., $y = 0$) indicates the presence (resp., absence) of the target property. A graph $G \in \mathcal{G}$ is given by $G = (V, E, A)$, where V is the node set, E is the edge set, and $A \in \{0, 1\}^{|V| \times |V|}$ is the adjacency matrix. Taking a perspective of statistical learning, each graph property detection task is associated with an unknown underlying distribution $\mathcal{D}$ over $\mathcal{G} \times \mathcal{Y}$. Given a training set $D = \{(G_i, y_i)\}_{i=1}^n \sim \mathcal{D}$, the goal is to design a hypothesis space $\mathcal{H} \subseteq \{h : \mathcal{G} \rightarrow \mathcal{Y}\}$ and learn a function $h \in \mathcal{H}$ that can minimize the true loss:

$$\mathcal{L}_{\mathcal{D}}(h) := \mathbb{E}_{(G, y) \sim \mathcal{D}} [\ell(h(G), y)],$$

where $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+$ measures the difference between the prediction $h(G)$ and the ground truth y .

4 Methodology

The pipeline of vision-based methods for graph property detection consists of three abstract steps.

- **Layout generation.** Given the input graph $G = (V, E, A) \in \mathcal{G}$, the generator produces a 2D layout based on the structural information of the graph.

$$\text{Generator} : \mathcal{G} \rightarrow \mathbb{R}^{|V| \times 2}.$$

- **Layout visualization.** The generated 2D graph layout is converted into an RGB image with a resolution of $H \times W$.

$$\text{Visualization} : \mathbb{R}^{|V| \times 2} \rightarrow \mathbb{R}^{H \times W \times 3}.$$

- **Image classification.** The RGB image is subsequently processed by an image classifier to detect the graph properties, outputting a binary label.

$$\text{Classifier} : \mathbb{R}^{H \times W \times 3} \rightarrow \{0, 1\}.$$

Our Vision neural Solver with Adaptive Layouts (VSAL) framework follows such a pipeline but with novel designs for layout generation and visualization. In what follows, we introduce each component of VSAL (Sec. 4.1), followed by its training methods (Sec. 4.2).

4.1 VSAL Framework

4.1.1 Layout Generation. In contrast to the current methods that utilize deterministic layouts, VSAL seeks to create flexible layouts that can be tuned to capture useful structural features for accurate detection. To this end, we design a class $\mathcal{G}_{\text{gen}}$ of random generators that produce graph layouts conditioned on a latent variable $\mathbf{z} \in \mathcal{Z} = \mathbb{R}^{d_z}$ of dimension $d_z \in \mathbb{Z}^+$, where each generator $g_{\theta_{\text{gen}}}$ is parameterized by a nominal set θ_{gen} of parameters:

$$\mathcal{G}_{\text{gen}} \subseteq \left\{ g_{\theta_{\text{gen}}}(G, \mathbf{z}) : \mathcal{G} \times \mathcal{Z} \rightarrow \mathbb{R}^{|V| \times 2} \right\}. \quad (1)$$

In particular, given an input graph $G = (V, E, A)$, its layout is generated by the following four steps: a) graph encoding, b) noise encoding, c) concatenation, and d) coordinate generation.

a) Graph encoding. The input adjacency matrix $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$ is processed through a graph encoder to extract structural features:

$$H_G = \text{GraphEncoder}_{\theta_{\text{gen}}^1}(\mathbf{A}) \in \mathbb{R}^{|V| \times d_g},$$

where $d_g \in \mathbb{Z}^+$ denotes the dimension of the hidden space and θ_{gen}^1 represents learnable parameters. We can adopt various graph representation models for the graph encoder, such as DenseGCN [21] and Graphormer [49].

b) Noise encoding. Simultaneously, a latent noise vector $\mathbf{z} \in \mathcal{Z}$ is sampled from a Gaussian distribution $P_{\mathcal{Z}} = \mathcal{N}(0, \sigma^2 \mathbf{I})$, where σ^2 is the variance and $\mathbf{I}$ represents the identity matrix. The noise vector $\mathbf{z}$ is then encoded into the same hidden space via a multilayer perceptron $\text{MLP}_{\theta_{\text{gen}}^2}$ with learnable parameters θ_{gen}^2 , introducing stochasticity to help generate diverse graph layouts [17].

$$H_z = \text{MLP}_{\theta_{\text{gen}}^2}(\mathbf{z}) \in \mathbb{R}^{|V| \times d_g}.$$

c) Concatenation. The graph embedding H_G is concatenated with the noise encoding H_z to form the node representation

$$\mathbf{H}_{\text{cond}} = \text{concat}(H_G, H_z) \in \mathbb{R}^{|V| \times 2d_g},$$

where the dimension of each node representation is $2d_g$.

d) Coordinates generation. Finally, $\mathbf{H}_{\text{cond}}$ is processed through a multilayer perceptron $\text{MLP}_{\theta_{\text{gen}}^3}$ parameterized by θ_{gen}^3 to compute the 2D coordinates for each node i :

$$(\hat{x}_i^1, \hat{x}_i^2) = \text{MLP}_{\theta_{\text{gen}}^3}(\mathbf{H}_{\text{cond}}(i)) \in \mathbb{R}^2.$$

In summary, the generator maps the input graph and the noise vector to the resulting graph layout $g_{\theta_{\text{gen}}}(G, \mathbf{z})$, i.e.,

$$g_{\theta_{\text{gen}}}(G, \mathbf{z}) = \left[(\hat{x}_1^1, \hat{x}_1^2), \dots, (\hat{x}_{|V|}^1, \hat{x}_{|V|}^2) \right] \in \mathbb{R}^{|V| \times 2}, \quad (2)$$

and the learnable parameters of the generator are collected as

$$\theta_{\text{gen}} = \left\{ \theta_{\text{gen}}^1, \theta_{\text{gen}}^2, \theta_{\text{gen}}^3 \right\}.$$

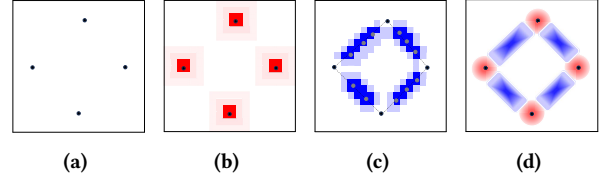


Figure 2: Layout visualization process: (a) normalized coordinates, (b) node rendering, (c) edge rendering, and (d) Gaussian smoothing.

4.1.2 Layout Visualization. In converting the generated layout $g_{\theta_{\text{gen}}}(G, \mathbf{z})$ into an RGB image, the coordinates $(\hat{x}_i^1, \hat{x}_i^2)$ of each node are first normalized to fit within the image resolution of $[0, H] \times [0, W]$ through

$$(\tilde{x}_i^1, \tilde{x}_i^2) = \left(\frac{\hat{x}_i^1 - \hat{x}_{\min}^1}{\hat{x}_{\max}^1 - \hat{x}_{\min}^1} \cdot H, \frac{\hat{x}_i^2 - \hat{x}_{\min}^2}{\hat{x}_{\max}^2 - \hat{x}_{\min}^2} \cdot W \right),$$

where $\hat{x}_{\min}^1, \hat{x}_{\max}^1, \hat{x}_{\min}^2$, and $\hat{x}_{\max}^2$ are the minimum and maximum values across all node coordinates in $g_{\theta_{\text{gen}}}(G, \mathbf{z})$. After that, an image tensor $\mathbf{M} \in \mathbb{R}^{H \times W \times 3}$ is initialized with all pixels set to (255, 255, 255). To ensure the differentiability of the visualization process, we circumvent the need for manual pixel coloring by using Gaussian falloff [12] to achieve smooth rendering. These visualization steps are illustrated in Fig. 2.

a) Node rendering. In visualizing the nodes, for each pixel $\mathbf{M}(p, q, :)$, its closeness to the nodes is measured by

$$\alpha_V(p, q) = 1 - \prod_{i=1}^{|V|} \left(1 - \exp \left(- \frac{d_i^2(p, q)}{2r^2} \right) \right),$$

where $d_i(p, q)$ is the Euclidean distance between (p, q) and $(\tilde{x}_i^1, \tilde{x}_i^2)$, and $r \in \mathbb{R}^+$ controls the degree of node influence. The RGB channel of $\mathbf{M}$ is then updated by

$$\mathbf{M}(p, q, :) = (1 - \alpha_V(p, q)) \cdot \mathbf{M}(p, q, :) + \alpha_V(p, q) \cdot (255, 0, 0).$$

Intuitively, the pixels closer to the node coordinates will be assigned with (255, 0, 0) of less transparency—Fig. 2b.

b) Edge rendering. Each edge $(i, j) \in E$ is associated with a set $L(i, j)$ of $N \in \mathbb{Z}^+$ points uniformly sampled along the line segment between nodes $(\tilde{x}_i^1, \tilde{x}_i^2)$ and $(\tilde{x}_j^1, \tilde{x}_j^2)$:

$$L(i, j) = \left\{ \frac{N+1-k}{N+1} (\tilde{x}_i^1, \tilde{x}_i^2) + \frac{k}{N+1} (\tilde{x}_j^1, \tilde{x}_j^2) \right\}_{k=1}^N.$$

Similarly, for each pixel $\mathbf{M}(p, q, :)$, its closeness to the edges is measured by

$$\alpha_E(p, q) = 1 - \prod_{(i, j) \in E} \left(1 - \exp \left(- \frac{d_{ij}^2(p, q)}{2\delta^2} \right) \right),$$

where $\delta \in \mathbb{R}^+$ controls the degree of edge influence, and $d_{ij}(p, q)$ is a smooth approximation of the Euclidean distance between (p, q) and $L(i, j)$, which is given by

$$d_{ij}(p, q) = \frac{\sum_{(x_1, x_2) \in L(i, j)} \exp \left(- \beta \cdot d_{x_1, x_2}^{p, q} \right) \cdot d_{x_1, x_2}^{p, q}}{\sum_{(x_1, x_2) \in L(i, j)} \exp \left(- \beta \cdot d_{x_1, x_2}^{p, q} \right)},$$

where $\beta \in \mathbb{R}^+$ controls the sharpness of approximation and $d_{x_1, x_2}^{p, q}$ is the Euclidean distance between (p, q) and $(x_1, x_2) \in L(i, j)$. The RGB channel of $\mathbf{M}$ is updated by

$$\mathbf{M}(p, q, :) = (1 - \alpha_E(p, q)) \cdot \mathbf{M}(p, q, :) + \alpha_E(p, q) \cdot (0, 0, 255),$$

with the implication that pixels closer to the edges will have a stronger color in $(0, 0, 255)$ —Fig. 2c.

c) Gaussian smoothing. Finally, we apply Gaussian smoothing to reduce pixelation and ensure smooth color transitions [42], which is implemented via

$$\mathbf{M}(p, q, :) = \sum_{u=-K}^K \sum_{v=-K}^K \text{Gau}(u, v) \cdot \mathbf{M}(p+u, q+v, :), \quad (3)$$

where $\text{Gau}(u, v) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2+v^2}{2\sigma^2}\right)$ denotes the Gaussian kernel with radius $K \in \mathbb{Z}^+$, and $\sigma \in \mathbb{R}^+$ controls the strength of the smoothing, as illustrated in Fig. 2d.

4.1.3 Image Classification and Wrap-up. The last step detects graph properties by an image classifier $f_{\theta_c}(\mathbf{M}) \in \{0, 1\}$, which can be, for example, ResNet-50 and ViT, with a set of learnable parameters denoted as θ_c . The proposed pipeline can be summarized as

$$\text{VSAL}_{\theta_{\text{gen}}, \theta_c}(G, \mathbf{z}) \in \{0, 1\}, \quad (4)$$

where it takes the graph G and the latent noise vector $\mathbf{z}$ as input with learnable parameters being $\{\theta_{\text{gen}}, \theta_c\}$.

4.2 Training Method

This section describes the training procedure of VSAL, i.e., deciding the best set of parameters $\{\theta_{\text{gen}}, \theta_c\}$. The parameter θ_c associated with the image classifier can be straightforwardly optimized by minimizing the classification loss, but finding the best θ_{gen} is less trivial. This is because the layouts from a random set θ_{gen} do not exhibit enough structural information for graph property detection. To address such issues, instead of training the generator from scratch, we seek to guide the generator to produce layouts that are similar to the given principled layouts (i.e., reference layouts), which is achieved through adversarial training; meanwhile, the parameters θ_{gen} are also optimized towards classification accuracy. In the rest of this section, we will first present the selected reference layouts and then describe the training method.

4.2.1 Reference Layout. In generating the reference layout of a given graph $G = (V, E, \mathbf{A})$, an initial layout $\mathbf{L}_{\text{initial}} \in \mathbb{R}^{|V| \times 2}$ is first created based on principled designs. We explore three principled designs and use the uniform layout as a baseline. For a graph G , the position of each node $i \in V$ is determined as follows.

- Circular [2]: The nodes are arranged uniformly on a circle of radius $r_1 \in \mathbb{R}^+$:

$$\mathbf{L}_{\text{initial}}[i, :] = (r_1 \cos(2\pi i/|V|), r_1 \sin(2\pi i/|V|)).$$

- Spiral [6]: The nodes are placed along an Archimedes spiral path parameterized by the offset factor $r_2 \in \mathbb{R}^+$:

$$\mathbf{L}_{\text{initial}}[i, :] = (i \cos(i \cdot r_2), i \sin(i \cdot r_2)).$$

- Shell [10]: The nodes are grouped into $S \in \mathbb{Z}^+$ concentric shells (circles), where shell j has radius $r_j \in \mathbb{R}^+$ and contains

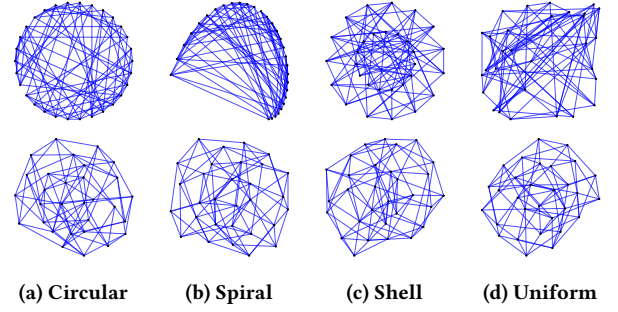


Figure 3: Each pair shows an example of the initial layout (top) and the refined layout (bottom) via Eq. 5.

n_j nodes such that $\sum_{j=1}^S n_j = |V|$:

$$\mathbf{L}_{\text{initial}}[i, :] = (r_j \cos(2\pi i/n_j), r_j \sin(2\pi i/n_j)).$$

- Uniform: The nodes are positioned randomly within a layout region bounded by $b \in \mathbb{R}^+$:

$$\mathbf{L}_{\text{initial}}[i, :] = (x_i, y_i), \quad x_i, y_i \sim U(-b, b).$$

To further refine the layout and reduce edge crossings, we apply a combination of the spring algorithm [15] and the Kamada-Kawai algorithm [27], of which the details can be found in Appendix A. The resulting reference layout is denoted as

$$\text{ref}(G) = \text{Kamada-Kawai}(\text{Spring}(\mathbf{L}_{\text{initial}})) \in \mathbb{R}^{|V| \times 2}. \quad (5)$$

Fig. 3 intuitively shows that this refinement can effectively reduce edge crossings and potentially improve the quality of initial layouts, thereby facilitating the training of the generator, as evidenced in our ablation studies (Sec. 5.2).

4.2.2 Adversarial Training with Classification Feedback. To encourage the generator to produce layouts that resemble reference layouts, we employ a Wasserstein adversarial training strategy [1]. To this end, we design a class $\mathcal{D}_{\text{dis}}$ of discriminators to distinguish between generated layouts $g_{\theta_{\text{gen}}}(G, \mathbf{z})$ (Eq. 2) and reference layouts $\text{ref}(G)$ (Eq. 5), where each discriminator $d_{\theta_{\text{dis}}}$ is parameterized by $\theta_{\text{dis}} = \{\theta_{\text{dis}}^1, \theta_{\text{dis}}^2\}$:

$$\mathcal{D}_{\text{dis}} \subseteq \{d_{\theta_{\text{dis}}}(\mathbf{L}, G = (V, E, \mathbf{A})) : \mathbb{R}^{|V| \times 2} \times \mathcal{G} \rightarrow \mathbb{R}\}.$$

In particular, the input graph layout $\mathbf{L}$ (either $g_{\theta_{\text{gen}}}(G, \mathbf{z})$ or $\text{ref}(G)$) and the adjacency matrix $\mathbf{A}$ are processed via a graph encoder, similar to the one used in the generator, to extract structural features.

$$H_D = \text{GraphEncoder}_{\theta_{\text{dis}}^1}(\mathbf{L}, \mathbf{A}) \in \mathbb{R}^{|V| \times d_s},$$

where $d_s \in \mathbb{Z}^+$ is the hidden dimension and θ_{dis}^1 is a set of learnable parameters. After that, a global average pooling layer is used to aggregate node features H_D into a graph-level representation.

$$\mathbf{h}_{\text{graph}} = \text{mean}(H_D) \in \mathbb{R}^{d_s}.$$

Finally, $\mathbf{h}_{\text{graph}}$ is passed through an MLP parameterized by θ_{dis}^2 to output the validity score, where higher scores indicate that the input layout $\mathbf{L}$ is more similar to the reference layout.

$$d_{\theta_{\text{dis}}}(\mathbf{L}, G) = \text{MLP}_{\theta_{\text{dis}}^2}(\mathbf{h}_{\text{graph}}) \in \mathbb{R}.$$

Now, we are ready to discuss the training process. Given the training set $\{(G_i, y_i)\}_{i=1}^n$, a collection $\{z_i^j\}_{j=1}^m$ of $m \in \mathbb{Z}^+$ noise vectors are sampled from P_Z for each pair (G_i, y_i) .

Training of θ_{gen} . The generator $g_{\theta_{\text{gen}}}$ is trained to produce graph layouts that can receive high scores from the discriminator $d_{\theta_{\text{dis}}}$ while also improving the detection effect. This is achieved by minimizing a combination of the empirical adversarial loss and classification loss:

$$\begin{aligned}\hat{\mathcal{L}}_g(\theta_{\text{gen}}, \theta_{\text{dis}}, \theta_c) &:= \hat{\mathcal{L}}_{\text{adv}}(\theta_{\text{gen}}, \theta_{\text{dis}}) + \lambda_c \hat{\mathcal{L}}_c(\theta_{\text{gen}}, \theta_c), \\ \hat{\mathcal{L}}_{\text{adv}}(\theta_{\text{gen}}, \theta_{\text{dis}}) &:= -\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m d_{\theta_{\text{dis}}}(g_{\theta_{\text{gen}}}(G_i, z_i^j), G_i), \\ \hat{\mathcal{L}}_c(\theta_{\text{gen}}, \theta_c) &:= \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m \ell_c(\text{VSAL}_{\theta_{\text{gen}}, \theta_c}(G_i, z_i^j), y_i),\end{aligned}$$

where ℓ_c is the cross-entropy loss function and $\lambda_c \in \mathbb{R}^+$ is a weight factor that balances these two loss components.

Training of θ_{dis} . The discriminator is trained to differentiate between generated layouts and reference layouts by optimizing the Wasserstein loss with gradient penalty [20]:

$$\begin{aligned}\hat{\mathcal{L}}_d(\theta_{\text{gen}}, \theta_{\text{dis}}) &:= -\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m d_{\theta_{\text{dis}}}(g_{\theta_{\text{gen}}}(G_i, z_i^j), G_i) \\ &\quad + \frac{1}{n} \sum_{i=1}^n d_{\theta_{\text{dis}}}(\text{ref}(G_i), G_i) + \lambda_{\text{gp}} \hat{\mathcal{L}}_{\text{gp}}(\theta_{\text{gen}}, \theta_{\text{dis}}),\end{aligned}$$

where $\lambda_{\text{gp}} \in \mathbb{R}^+$ is the gradient penalty coefficient. The gradient penalty term $\hat{\mathcal{L}}_{\text{gp}}$ enforces the 1-Lipschitz constraint to prevent vanishing or exploding gradients in the discriminator, ensuring that the generator receives consistently effective guidance [1, 20]:

$$\begin{aligned}\hat{\mathcal{L}}_{\text{gp}}(\theta_{\text{gen}}, \theta_{\text{dis}}) &:= \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m (\|\nabla_{\hat{\mathbf{L}}_i^j} d_{\theta_{\text{dis}}}(\hat{\mathbf{L}}_i^j, G_i)\|_2 - 1)^2, \\ \hat{\mathbf{L}}_i^j &:= \lambda \text{ref}(G_i) + (1 - \lambda) g_{\theta_{\text{gen}}}(G_i, z_i^j), \lambda \sim U(0, 1),\end{aligned}$$

where $\hat{\mathbf{L}}_i^j$ is the interpolation between the reference layout $\text{ref}(G_i)$ and the generated layout $g_{\theta_{\text{gen}}}(G_i, z_i^j)$.

Training of θ_c . The classifier is trained by minimizing the classification loss $\hat{\mathcal{L}}_c(\theta_{\text{gen}}, \theta_c)$.

Optimization scheme. To stabilize VSAL training for better detection accuracy, we pretrain θ_{gen} and θ_{dis} to mitigate the instability of adversarial training [45], which is done by

$$\frac{\partial \hat{\mathcal{L}}_{\text{adv}}(\theta_{\text{gen}}, \theta_{\text{dis}})}{\partial \theta_{\text{gen}}} \text{ and } \frac{\partial \hat{\mathcal{L}}_d(\theta_{\text{gen}}, \theta_{\text{dis}})}{\partial \theta_{\text{dis}}}. \quad (6)$$

This pretraining enables the generator to learn the fundamental principles for generating structured layouts, while ensuring that the discriminator can effectively differentiate reference layouts from poorly structured ones, thus providing a stable foundation for the training of VSAL. After pretraining, we iteratively optimize the parameters of each component through coordinated gradient descent via

$$\frac{\partial \hat{\mathcal{L}}_g(\theta_{\text{gen}}, \theta_{\text{dis}}, \theta_c)}{\partial \theta_{\text{gen}}}, \frac{\partial \hat{\mathcal{L}}_d(\theta_{\text{gen}}, \theta_{\text{dis}})}{\partial \theta_{\text{dis}}}, \text{ and }, \frac{\partial \hat{\mathcal{L}}_c(\theta_{\text{gen}}, \theta_c)}{\partial \theta_c}.$$

Table 1: Statistics of datasets. Graph size: small (4-20 nodes), medium (21-50 nodes), large (401-500 nodes), and huge (901-1000 nodes). “T” and “F” denote true and false, respectively.

	House of Graphs				Synthetic			
	Small		Medium		Large		Huge	
	T	F	T	F	T	F	T	F
Ham	2277	1838	7453	5739	2000	2000	2000	2000
Planar	1947	2000	1999	2000	2000	2000	2000	2000
Claw	919	946	-	-	2000	2000	2000	2000
Tree	213	481	-	-	2000	2000	2000	2000

5 Experiments

In this section, we evaluate the effectiveness of our VSAL framework and demonstrate that VSAL behaves in the designed way.

5.1 Settings

5.1.1 Tasks and Datasets. We consider four well-known graph property detection tasks: Hamiltonian cycle problem (**Ham**) [18], planarity verification (**Planar**) [25], claw-free graph classification (**Claw**) [14], and tree recognition (**Tree**) [40]; their formal definitions can be found in Appendix B.1. We conduct experiments on datasets of varying graph sizes. The small and medium datasets are borrowed from the House of Graphs [9], while the large and huge datasets are synthetically produced, as described in Appendix B.2. The summary statistics for all datasets are provided in Table 1.

5.1.2 VSAL Settings. To comprehensively evaluate VSAL, we explore multiple configurations of graph encoders, backbone classifiers, and input resolutions. Specifically, we implement VSAL with two graph encoders: DenseGCN and Graphormer. As for image classification, we adopt three widely used backbone classifiers: ResNet-50, Vision Transformer (ViT), and EfficientNet [44], all initialized with ImageNet-pretrained weights [11, 22]. For ResNet-50 and ViT, we use an input resolution of 224×224 . For EfficientNet, we evaluate three resolutions: 224×224 , 380×380 , and 528×528 . We denote these model variants as **VSAL-R-224**, **VSAL-V-224**, **VSAL-E-224**, **VSAL-E-380**, and **VSAL-E-528**, respectively.

5.1.3 Baseline Methods. We compare VSAL with the following baseline methods, which have been introduced in Sec. 2.

- **VN-Solver** [38]: We follow the official implementation and use both circular and spiral layouts with a ResNet-50 backbone at 224×224 resolution.
- **Graphormer** [49]: We use the official pretrained checkpoint pcqm4mv2_graphormer_base, configured with 768 hidden units and 12 attention heads, and conduct fine-tuning.
- **Graphormer-GD** [50]: We train the model from scratch following the official codebase.
- **EquiformerV2** [32]: We load the publicly released checkpoint eq2_83M_2M.pt and fine-tune it on our datasets.
- **GraphsGPT** [16]: We adopt the official released encoder checkpoint GraphsGPT-4W and fine-tune it for our tasks.
- **Random**: A baseline assigning labels uniformly at random.

Table 2: F1 scores of all models on four graph property detection tasks. Each model is trained with a size of 200 samples on small tree graphs and 1000 samples on all other datasets. VSAL is implemented with two representative graph encoders: DenseGCN and Graphormer. All VSAL variants use circular initial layouts. VN-Solver-C and VN-Solver-S denote VN-Solver with circular and spiral layouts, respectively. The top three results for each task are highlighted.

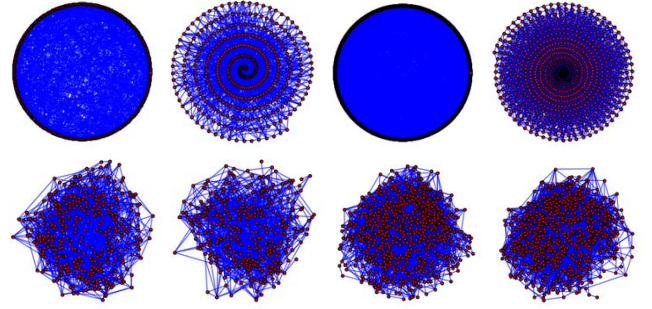
		Ham				Planar				Claw			Tree		
		Small	Medium	Large	Huge	Small	Medium	Large	Huge	Small	Large	Huge	Small	Large	Huge
VSAL	DenseGCN R-224	0.89 _(0.01)	0.96_(0.01)	0.89 _(0.00)	0.89 _(0.01)	0.86 _(0.01)	0.97_(0.01)	0.91 _(0.01)	0.89 _(0.01)	0.91 _(0.01)	0.91 _(0.01)	0.90 _(0.01)	0.93 _(0.01)	0.91 _(0.01)	0.90 _(0.01)
	DenseGCN V-224	0.90_(0.01)	0.98_(0.00)	0.91 _(0.00)	0.90 _(0.01)	0.89 _(0.01)	0.97_(0.00)	0.92 _(0.01)	0.91 _(0.01)	0.93 _(0.01)	0.93_(0.01)	0.92_(0.01)	0.96_(0.01)	0.94 _(0.01)	0.92 _(0.00)
	DenseGCN E-224	0.89 _(0.01)	0.97_(0.00)	0.89 _(0.01)	0.88 _(0.01)	0.87 _(0.01)	0.97_(0.00)	0.91 _(0.01)	0.90 _(0.01)	0.92 _(0.01)	0.91 _(0.01)	0.91 _(0.01)	0.96_(0.01)	0.91 _(0.01)	0.90 _(0.01)
	DenseGCN E-380	0.90 _(0.01)	0.97_(0.01)	0.92 _(0.00)	0.91_(0.00)	0.87 _(0.01)	0.97_(0.00)	0.93_(0.01)	0.91 _(0.01)	0.92 _(0.01)	0.92_(0.01)	0.91 _(0.00)	0.96_(0.01)	0.93 _(0.01)	0.93_(0.01)
	DenseGCN E-528	0.91_(0.01)	0.98_(0.01)	0.94_(0.01)	0.92_(0.00)	0.87 _(0.01)	0.98_(0.01)	0.93_(0.00)	0.92_(0.01)	0.93 _(0.01)	0.93_(0.01)	0.92_(0.01)	0.96_(0.01)	0.95_(0.01)	0.94_(0.01)
	Graphormer R-224	0.93_(0.01)	0.97_(0.00)	0.93_(0.01)	0.90 _(0.01)	0.90_(0.01)	0.97_(0.01)	0.92 _(0.00)	0.91 _(0.01)	0.94_(0.01)	0.93_(0.01)	0.91 _(0.01)	0.97_(0.01)	0.95_(0.01)	0.92 _(0.01)
	Graphormer V-224	0.93_(0.01)	0.98_(0.00)	0.94_(0.01)	0.92_(0.01)	0.92_(0.01)	0.98_(0.00)	0.94_(0.01)	0.93_(0.01)	0.95_(0.01)	0.95_(0.01)	0.94_(0.01)	0.98_(0.01)	0.96_(0.01)	0.94_(0.01)
	Graphormer E-224	0.93_(0.01)	0.98_(0.01)	0.93_(0.01)	0.91_(0.01)	0.91_(0.01)	0.98_(0.00)	0.93_(0.01)	0.92_(0.01)	0.94_(0.00)	0.93_(0.01)	0.92_(0.01)	0.98_(0.01)	0.95_(0.00)	0.93_(0.01)
	Graphormer E-380	0.93_(0.01)	0.98_(0.00)	0.94_(0.00)	0.93_(0.01)	0.91_(0.01)	0.98_(0.00)	0.94_(0.01)	0.92_(0.01)	0.95_(0.01)	0.93_(0.01)	0.93_(0.01)	0.98_(0.00)	0.96_(0.01)	0.94_(0.01)
	Graphormer E-528	0.94_(0.00)	0.98_(0.01)	0.95_(0.01)	0.93_(0.00)	0.92_(0.01)	0.99_(0.01)	0.95_(0.00)	0.94_(0.01)	0.96_(0.01)	0.95_(0.00)	0.94_(0.01)	0.98_(0.00)	0.97_(0.01)	0.95_(0.01)
VN-Solver-C [38]		0.83 _(0.03)	0.95 _(0.01)	0.57 _(0.03)	0.53 _(0.01)	0.85 _(0.02)	0.95 _(0.01)	0.51 _(0.02)	0.51 _(0.01)	0.86 _(0.01)	0.60 _(0.01)	0.58 _(0.02)	0.83 _(0.02)	0.76 _(0.01)	0.64 _(0.02)
VN-Solver-S [38]		0.80 _(0.02)	0.95 _(0.01)	0.81 _(0.01)	0.76 _(0.01)	0.84 _(0.01)	0.94 _(0.02)	0.53 _(0.01)	0.51 _(0.02)	0.85 _(0.01)	0.68 _(0.02)	0.62 _(0.02)	0.81 _(0.03)	0.77 _(0.02)	0.68 _(0.02)
Graphormer [49]		0.88 _(0.00)	0.93 _(0.00)	0.88 _(0.00)	0.80 _(0.00)	0.88 _(0.00)	0.96 _(0.00)	0.90 _(0.00)	0.83 _(0.00)	0.89 _(0.00)	0.80 _(0.00)	0.69 _(0.00)	0.98_(0.00)	0.87 _(0.00)	0.79 _(0.00)
Graphormer-GD [50]		0.88 _(0.01)	0.95 _(0.01)	0.90 _(0.00)	0.82 _(0.01)	0.89 _(0.01)	0.95 _(0.01)	0.89 _(0.01)	0.82 _(0.01)	0.90 _(0.02)	0.83 _(0.02)	0.71 _(0.01)	0.98_(0.00)	0.88 _(0.01)	0.81 _(0.01)
EquiformerV2 [32]		0.85 _(0.01)	0.92 _(0.01)	0.89 _(0.01)	0.79 _(0.01)	0.83 _(0.00)	0.94 _(0.00)	0.85 _(0.01)	0.79 _(0.02)	0.85 _(0.01)	0.77 _(0.02)	0.67 _(0.02)	0.94 _(0.00)	0.85 _(0.02)	0.77 _(0.01)
GraphsGPT [16]		0.87 _(0.00)	0.94 _(0.01)	0.90 _(0.01)	0.81 _(0.01)	0.90_(0.00)	0.97_(0.00)	0.91 _(0.01)	0.82 _(0.02)	0.92 _(0.01)	0.81 _(0.01)	0.73 _(0.02)	0.98_(0.00)	0.89 _(0.01)	0.81 _(0.02)
Random		0.53 _(0.02)	0.53 _(0.02)	0.51 _(0.01)	0.51 _(0.01)	0.49 _(0.02)	0.49 _(0.02)	0.51 _(0.01)	0.51 _(0.01)	0.50 _(0.01)	0.51 _(0.01)	0.51 _(0.01)	0.38 _(0.02)	0.51 _(0.01)	0.51 _(0.01)

5.1.4 Training Setting. The considered learning models are trained for up to 100 epochs with early stopping based on validation accuracy. The training and testing samples are randomly selected from the datasets, with the training size selected from {200, 1000}. For datasets from the House of Graphs, the testing size is 500 (resp., 200) for Ham, Planar, and Claw (resp., Tree), while for synthetic datasets, the test set size is fixed at 500 for all tasks. Each training set is further split into 80% for training and 20% for validation. As for the adversarial pretraining step (i.e., Eq. 6), a separate pre-training set is randomly sampled from the remaining data: 2000 samples for Ham and Planar, 1000 for Claw, and 200 for Tree, using datasets from House of Graphs; and 2000 samples per task for the synthetic datasets. Each method is repeated five times with different random seeds, and we report the average F1 score and standard deviations. Detailed training settings can be found in Appendix B.3, including hyperparameter settings, learning rates, optimizers, and configuration of computing platforms.

5.2 Experimental Results Analysis

In this section, we analyze the comparative results and also present studies to verify that VSAL indeed operates as expected. The main results are summarized in Table 2, where all VSAL variants use circular initial layouts. The complete results, including those of VSAL with different initial layouts, as well as additional analysis, are provided in Appendix B.5.

VSAL outperforms VN-Solver. VSAL consistently achieves better results over VN-Solver, which relies on fixed layouts, by an evident margin, demonstrating the effectiveness of adaptive layout generation; for example, VSAL-V with Graphormer improves the F1



(a) large circular (b) large spiral (c) huge circular (d) huge spiral

Figure 4: Each pair illustrates graph layouts produced by VN-Solver (top) and VSAL (bottom) on the Ham. (a) and (b) show large graph layouts with circular and spiral layouts. (c) and (d) present the corresponding layouts for huge graphs.

score from 0.76 to 0.92 on the Ham with huge graphs and from 0.83 to 0.98 on the Tree with small graphs. Notably, VSAL still maintains strong performance on large and huge graphs. By comparison, the results of VN-Solver heavily rely on the choice of fixed layouts; for instance, on the Ham with large and huge graphs, circular layouts approach random guessing, whereas spiral layouts yield effective results. This reveals that different fixed layouts may only work under specific scenarios. As illustrated in Fig. 4, fixed circular layouts often lead to severe edge overlaps in larger graphs, obscuring structural features, while spiral layouts can retain partial useful

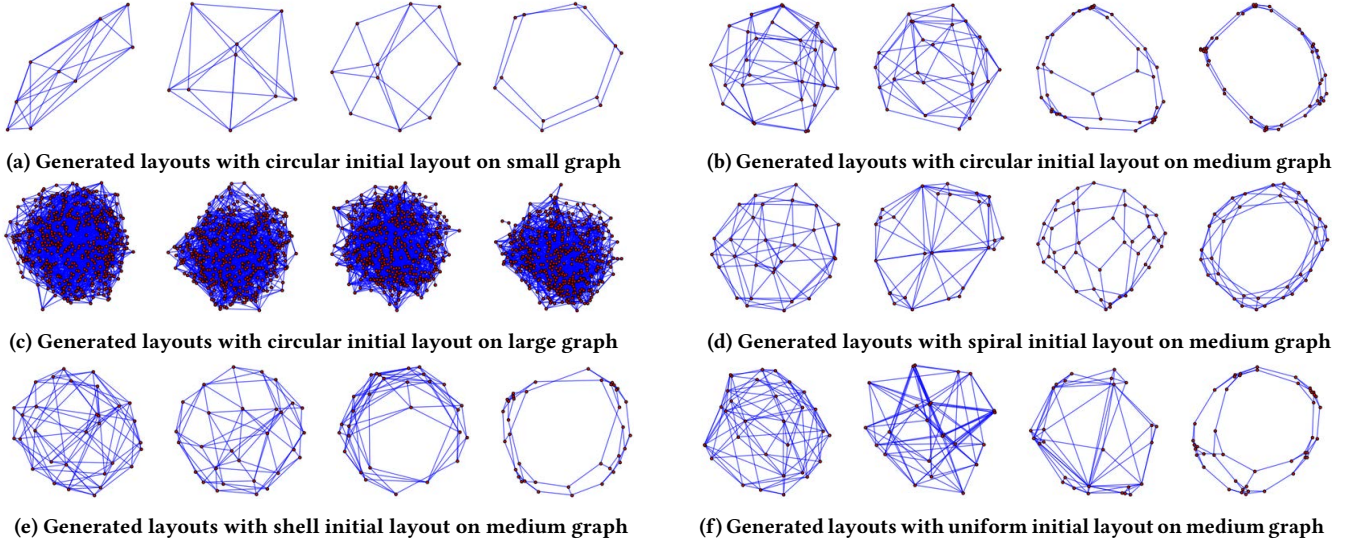


Figure 5: Examples of graph layouts generated during the training of VSAL on the Hamiltonian cycle problem at a resolution of 224×224 . Each group shows four layouts generated after, respectively, 0, 5, 10, and 15 epochs, from left to right.

Table 3: F1 scores of different initial layouts on the Ham.

Train size 1000	VSAL-V-224		VN-Solver	
	Small	Medium	Small	Medium
Circular	0.90 _(0.01)	0.98 _(0.00)	0.83 _(0.03)	0.95 _(0.01)
Spiral	0.90 _(0.01)	0.98 _(0.01)	0.80 _(0.02)	0.95 _(0.01)
Shell	0.89 _(0.01)	0.97 _(0.00)	0.74 _(0.05)	0.94 _(0.02)
Uniform	0.87 _(0.01)	0.96 _(0.01)	0.58 _(0.01)	0.65 _(0.00)

features. In contrast, the learned layouts of VSAL exhibit reduced edge overlaps and better preserve key features for detection.

VSAL maintains robustness across various initial layouts.

Table 3 presents the results of DenseGCN-based VSAL and VN-Solver using different initial layouts, i.e., circular (Fig. 5b), spiral (Fig. 5d), shell (Fig. 5e), and uniform (Fig. 5f), on the Ham; complete results for other tasks are provided in Table 7 (Appendix B.5). As expected, structured initial layouts with clear spatial organization provide more informative visual representations for detection than uniform layouts. Interestingly, from Table 3, VSAL maintains high F1 scores across different initial layouts including uniform ones, which proves its ability to learn to generate effective layouts even from less principled initial layouts. For instance, VSAL-V achieves F1 scores ranging from 0.96 to 0.98 across different initial layouts on medium graphs. In contrast, the performance of VN-Solver drops sharply with less structured layouts; for example, from 0.95 (circular and spiral) to 0.65 (uniform) on medium graphs.

VSAL-generated layouts enhance clarity and capture useful features. Fig. 5 illustrates the dynamic changes of generated layouts throughout training on the Ham. For small and medium graphs, the layouts exhibit increasing structural clarity with fewer edge crossings, offering more useful features that facilitate the detection of the Hamiltonian cycle. In the case of large and huge

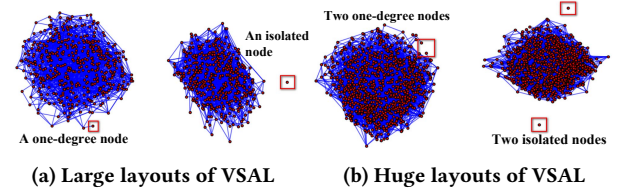


Figure 6: VSAL-generated layouts on the Ham highlight structural features critical for prediction. In these layouts, one-degree nodes are distinctly positioned and isolated nodes are clearly separated, thus providing visual cues that help the model better infer the absence of Hamiltonian cycles.

graphs, the relatively small image resolution constrains structural clarity (Fig. 5c); nevertheless, the generated layouts can still emphasize key visual features. For example, as shown in Fig. 6, the generated layouts on the Ham effectively capture visual cues such as isolated nodes and one-degree nodes—both critical for the Hamiltonian cycle problem. Such observations demonstrate that VSAL effectively learns to generate graph layouts in an expected manner and underscore the significance of incorporating classifier feedback into the training of the generator.

VSAL with Graphormer encoder outperforms VSAL with DenseGCN. We assess VSAL using two representative graph encoders: Graphormer and DenseGCN. As reported in Table 2, the Graphormer-based variant consistently achieves higher F1 scores across multiple tasks. For instance, on the Ham with small graphs, VSAL-V with Graphormer reaches an F1 score of 0.93, compared to 0.90 with DenseGCN. This improvement aligns with our expectation, as Graphormer can capture long-range dependencies and global structural features effectively, offering stronger expressive power than DenseGCN. This enables VSAL to generate more informative layouts and achieve higher classification accuracy.

Table 4: Ablation results (F1 scores) of Graphormer-based VSAL-V-224. “S” and “M” denotes small and medium graphs. Exp-A: w/o Pretraining, Exp-B: w/o Refinement, Exp-C: w/o Gaussian Smoothing, Exp-D: w/o Visualization Module.

		VSAL-V	Exp-A	Exp-B	Exp-C	Exp-D
Ham	S	0.93 _(0.01)	0.88 _(0.01)	0.89 _(0.01)	0.91 _(0.01)	0.90 _(0.00)
	M	0.98 _(0.00)	0.94 _(0.01)	0.95 _(0.01)	0.97 _(0.01)	0.96 _(0.00)
Planar	S	0.92 _(0.01)	0.87 _(0.02)	0.88 _(0.01)	0.90 _(0.01)	0.89 _(0.01)
	M	0.98 _(0.00)	0.95 _(0.01)	0.94 _(0.01)	0.96 _(0.01)	0.95 _(0.01)
Claw	S	0.95 _(0.01)	0.90 _(0.02)	0.91 _(0.01)	0.93 _(0.01)	0.92 _(0.00)
Tree	S	0.98 _(0.01)	0.93 _(0.02)	0.93 _(0.01)	0.95 _(0.01)	0.94 _(0.01)

VSAL-V outperforms VSAL-R and VSAL-E at the same resolution. While VSAL-V, VSAL-R, and VSAL-E all achieve strong results, VSAL-V shows a slight advantage, particularly on small graphs (see Table 2). For instance, on the Planar with medium graphs and 224×224 resolution, all three DenseGCN-based variants achieve the same F1 scores of 0.97. However, when the graph size is changed to small, VSAL-V outperforms others, achieving an F1 score of 0.89 compared to 0.86 for VSAL-R and 0.87 for VSAL-E. The results echo the fact that ViT is in general more effective in capturing visual patterns than ResNet-50 and EfficientNet.

Higher resolution achieves better performance. We evaluate VSAL at multiple image resolutions and observe that resolution influences detection accuracy, especially for large and huge graphs. As shown in Table 2, resolution has little impact on small and medium graphs, where the lower density of nodes and edges enables visual clarity even at limited resolutions. In contrast, large and huge graphs exhibit increased visual clutter at low resolutions, and higher resolutions preserve fine-grained structural details that enhance detection. For instance, on the Ham with large graphs, DenseGCN-based VSAL-E achieves an F1 score of 0.94 at 528×528 resolution, compared to 0.92 at 380×380 and 0.89 at 224×224 .

The impact of critical modules in VSAL. We conduct ablation studies through the following experiments using Graphormer-based VSAL-V: (Exp-A) VSAL-V without pretraining (Eq. 6), (Exp-B) VSAL-V without the refinement module (Eq. 5), (Exp-C) VSAL-V without Gaussian smoothing (Eq. 3), and (Exp-D) VSAL-V without the visualization module (Sec. 4.1.2). The first three cases ablate individual modules, whereas Exp-D removes the entire visualization module and instead employs a customized Graphormer classifier to directly process the generated layouts for label prediction. The results in Table 4 indicate that all modules contribute to improved detection performance. Notably, the degradation in Exp-D confirms the necessity of the visualization module in VSAL. Additional results on large and huge graphs are provided in Appendix B.5.

VSAL outperforms matrix-based methods. As shown in Table 2, VSAL surpasses state-of-the-art matrix-based methods, including Graphormer, Graphormer-GD, EquiformerV2, and GraphsGPT on most tasks, particularly on the large and huge graphs, and achieves comparable performance in the remaining cases. For example, on the Claw with large graphs, DenseGCN-based VSAL-V achieves an F1 score of 0.93, outperforming Graphormer (0.80), Graphormer-GD (0.83), EquiformerV2 (0.77), and GraphsGPT (0.81).

These results confirm again the effectiveness of vision-based approaches, which not only achieve promising accuracy but also offer improved interpretability through explicit layout visualization.

VSAL achieves higher efficiency in terms of inference time and memory usage. We compare the inference time of DenseGCN-based VSAL with several exact methods on different tasks, as detailed in Appendix B.5. The results in Table 8 (Appendix B.5) demonstrate that VSAL can achieve extreme efficiency without sacrificing much efficacy; for instance, VSAL-V processes a single huge graph in just 0.015 seconds, whereas the Held-Karp algorithm [23] takes over 3600 seconds on the Ham. Additionally, we compare GPU memory usage between DenseGCN-based VSAL and Graphormer, as shown in Table 9 (Appendix B.5). With graph size increasing, VSAL maintains significantly lower memory usage; for instance, 2965.8 MB on huge graphs compared to 39785.8 MB of Graphormer. This is because the memory usage of VSAL depends on image resolution (set to 224×224 here, but adjustable), while Graphormer scales with adjacency matrix size (1000×1000 for huge graphs). Although image representations may abstract some details, their resolution can be tuned to balance efficiency and accuracy, thus enabling VSAL to process much larger graphs under limited computational resources.

6 Future Discussion

In this paper, we propose VSAL, a new vision-based framework that can generate adaptive layouts for various graph property detection tasks. We conclude by offering several interesting directions:

Task-specific vision-based methods. While VSAL achieves better results than several advanced general-purpose matrix-based methods such as Graphormer and GraphsGPT, it does not rule out the scenario that VSAL can be less effective than matrix-based methods tailored to specific tasks [43]. Future work could incorporate task-specific features to enhance detection, such as explicitly visualizing potential cycle structures for Hamiltonian cycles.

Exploring visualization methods. Technically, any mapping $\mathcal{G} \rightarrow \mathbb{R}^{H \times W \times 3}$ can serve as the graph visualization module, leaving great room to explore more effective visualization strategies. A direction is to investigate how visual attributes, such as node size, edge thickness, and color schemes, affect detection performance. For example, enlarging or recoloring isolated nodes (as in Fig. 6) may help Hamiltonian cycle detection. Another direction is to design more efficient and fully differentiable visualization methods, such as differentiable rasterization, to better integrate layout generation and classification while reducing computational overhead.

Advanced generative models and alternative representation methods. Advanced generative models (e.g., diffusion models [24]) may offer higher generative quality, and therefore, integrating such methods into VSAL may lead to better generalization performance. The challenge in doing so is that training these models results in substantial computational overhead. Finally, enhancing the generator with more effective representations (e.g., incorporating subgraph embeddings) may improve its expressiveness and enable the generation of more informative layouts.

Acknowledgments

This project is supported in part by National Science Foundation under IIS-2144285 and IIS-2414308.

References

- [1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein generative adversarial networks. In *International conference on machine learning*. PMLR, Sydney, NSW, Australia, 214–223.
- [2] Swati A Bhavsar, Varsha H Patil, and Aboli H Patil. 2022. Graph partitioning and visualization in graph mining: a survey. *Multimedia tools and applications* 81, 30 (2022), 43315–43356.
- [3] Leonora Bianchi, Marco Dorigo, Luca Maria Gambardella, and Walter J Gutjahr. 2009. A survey on metaheuristics for stochastic combinatorial optimization. *Natural Computing* 8 (2009), 239–287.
- [4] Andrei Broder, Ravi Kumar, Farzin Maghoul, Prabhakar Raghavan, Sridhar Rajagopalan, Raymie Stata, Andrew Tomkins, and Janet Wiener. 2000. Graph structure in the web. *Computer networks* 33, 1–6 (2000), 309–320.
- [5] Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Velicković. 2023. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research* 24, 130 (2023), 1–61.
- [6] John V Carlis and Joseph A Konstan. 1998. Interactive visualization of serial periodic data. In *Proceedings of the 11th annual ACM symposium on User interface software and technology*. ACM, New York, NY, USA, 29–38.
- [7] S Jeremy Carriere and Rick Kazman. 1997. WebQuery: Searching and visualizing the Web through connectivity. *Computer Networks and ISDN Systems* 29, 8–13 (1997), 1257–1267.
- [8] Can Chen and Yang-Yu Liu. 2023. A survey on hyperlink prediction. *IEEE Transactions on Neural Networks and Learning Systems* 35, 11 (2023), 15034–15050.
- [9] Kris Coolsaet, Sven D'hondt, and Jan Goedgebeur. 2023. House of Graphs 2.0: A database of interesting graphs and more. *Discrete Applied Mathematics* 325 (2023), 97–107.
- [10] Josep Diaz, Jordi Petit, and Maria Serna. 2002. A survey of graph layout problems. *ACM Computing Surveys (CSUR)* 34, 3 (2002), 313–356.
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.
- [12] Iddo Drori, Daniel Cohen-Or, and Hezy Yeshurun. 2003. Fragment-based image completion. *ACM Transactions on Graphics* 22, 3 (2003), 303–312.
- [13] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yohua Bengio, and Xavier Bresson. 2023. Benchmarking graph neural networks. *Journal of Machine Learning Research* 24, 43 (2023), 1–48.
- [14] Ralph Faudree, Evelyn Flandrin, and Zdeněk Ryjáček. 1997. Claw-free graphs—a survey. *Discrete Mathematics* 164, 1–3 (1997), 87–147.
- [15] Thomas MJ Fruchterman and Edward M Reingold. 1991. Graph drawing by force-directed placement. *Software: Practice and experience* 21, 11 (1991), 1129–1164.
- [16] Zhangyang Gao, Daize Dong, Cheng Tan, Jun Xia, Bozhen Hu, and Stan Z Li. 2024. A Graph is Worth K Words: Euclideanizing Graph using Pure Transformer. In *International Conference on Machine Learning*. PMLR, Vienna, Austria, 14681–14701.
- [17] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. *Advances in neural information processing systems* 27 (2014), 2672–2680.
- [18] Ronald J Gould. 2014. Recent advances on the Hamiltonian problem: Survey III. *Graphs and Combinatorics* 30 (2014), 1–46.
- [19] Alexandros Graikos, Nikolay Malkin, Nebojsa Jovic, and Dimitris Samaras. 2022. Diffusion models as plug-and-play priors. *Advances in Neural Information Processing Systems* 35 (2022), 14715–14728.
- [20] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. 2017. Improved training of wasserstein GANs. *Advances in neural information processing systems* 30 (2017), 5769–5779.
- [21] Zhijiang Guo, Yan Zhang, Zhiyang Teng, and Wei Lu. 2019. Densely Connected Graph Convolutional Networks for Graph-to-Sequence Learning. *Transactions of the Association for Computational Linguistics* 7 (March 2019), 297–312.
- [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. IEEE, USA, 770–778.
- [23] Michael Held and Richard M Karp. 1962. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied mathematics* 10, 1 (1962), 196–210.
- [24] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [25] John Hopcroft and Robert Tarjan. 1974. Efficient planarity testing. *Journal of the ACM (JACM)* 21, 4 (1974), 549–568.
- [26] Nitin Jindal and Bing Liu. 2007. Review spam detection. In *Proceedings of the 16th international conference on World Wide Web*. ACM, New York, NY, USA, 1189–1190.
- [27] Tomihisa Kamada, Satoru Kawai, et al. 1989. An algorithm for drawing general undirected graphs. *Information processing letters* 31, 1 (1989), 7–15.
- [28] Wouter Kool, Herke van Hoof, and Max Welling. 2019. Attention, Learn to Solve Routing Problems!. In *International Conference on Learning Representations*.
- [29] Oh-Hyun Kwon and Kwan-Liu Ma. 2020. A deep generative model for graph layout. *IEEE Transactions on visualization and Computer Graphics* 26, 1 (2020), 665–675.
- [30] Jure Leskovec, Kevin J Lang, and Michael Mahoney. 2010. Empirical comparison of algorithms for network community detection. In *Proceedings of the 19th international conference on World wide web*. ACM, New York, NY, USA, 631–640.
- [31] Jiakang Li, Songning Lai, Zhihao Shuai, Yuan Tan, Yifan Jia, Mianyang Yu, Zichen Song, Xiaokang Peng, Ziyang Xu, Yongxin Ni, et al. 2024. A comprehensive review of community detection in graphs. *Neurocomputing* 600 (2024), 128169.
- [32] Yi-Lun Liao, Brandon M Wood, Abhishek Das, and Tess Smidt. 2024. EquiformerV2: Improved Equivariant Transformer for Scaling to Higher-Degree Representations. In *International Conference on Learning Representations*.
- [33] Zhengxuan Ling, Xinyu Tao, Yu Zhang, and Xi Chen. 2020. Solving optimization problems through fully convolutional networks: An application to the traveling salesman problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 51, 12 (2020), 7475–7485.
- [34] Zhengxuan Ling, Yu Zhang, and Xi Chen. 2023. A deep reinforcement learning based real-time solution policy for the traveling salesman problem. *IEEE Transactions on Intelligent Transportation Systems* 24, 6 (2023), 5871–5882.
- [35] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2021), 12012–12038.
- [36] Robert Meusel, Sebastiano Vigna, Oliver Lehmeberg, and Christian Bizer. 2014. Graph structure in the web—revisited: a trick of the heavy tail. In *Proceedings of the 23rd international conference on World Wide Web*. ACM, New York, NY, USA, 427–432.
- [37] Seth A Myers, Aneesh Sharma, Pankaj Gupta, and Jimmy Lin. 2014. Information network or social network? The structure of the Twitter follow graph. In *Proceedings of the 23rd international conference on world wide web*. ACM, New York, NY, USA, 493–498.
- [38] Mina Samizadeh and Guangmo Tong. 2023. VN-Solver: Vision-based Neural Solver for Combinatorial Optimization over Graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. ACM, New York, NY, USA, 4269–4273.
- [39] Martin JA Schuetz, J Kyle Brubaker, and Helmut G Katzgraber. 2022. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence* 4, 4 (2022), 367–377.
- [40] Dennis Shasha, Jason TL Wang, and Rosalba Giugno. 2002. Algorithmics and applications of tree and graph searching. In *Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. ACM, New York, NY, USA, 39–52.
- [41] Ben Shneiderman and Aleks Aris. 2006. Network visualization by semantic substrates. *IEEE transactions on visualization and computer graphics* 12, 5 (2006), 733–740.
- [42] Patrice Y Simard, Dave Steinkraus, and John C Platt. 2003. Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, Vol. 2. IEEE, USA, 958.
- [43] Zhiqing Sun and Yiming Yang. 2023. Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in neural information processing systems* 36 (2023), 3706–3731.
- [44] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, Long Beach, California, USA, 6105–6114.
- [45] Hoang Thanh-Tung, Truyen Tran, and Svetha Venkatesh. 2019. Improving Generalization and Stability of Generative Adversarial Networks. In *International Conference on Learning Representations*.
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017), 6000–6010.
- [47] Xiaoqi Wang, Kevin Yen, Yifan Hu, and Han-Wei Shen. 2023. SmartGD: A GAN-Based graph drawing framework for diverse aesthetic goals. *IEEE Transactions on Visualization and Computer Graphics* 30, 8 (2023), 5666–5678.
- [48] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [49] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in neural information processing systems* 34 (2021), 28877–28888.
- [50] Bohang Zhang, Shengjie Luo, Liwei Wang, and Di He. 2023. Rethinking the Expressive Power of GNNs via Graph Biconnectivity. In *International Conference on Learning Representations*.

Appendix

A Spring and Kamada-Kawai Algorithms

In this section, we describe the two layout algorithms employed in our framework: the Spring algorithm [15] and the Kamada-Kawai algorithm [27]. The pseudocode for the Spring layout algorithm is provided in Algorithm 1, while the pseudocode for the Kamada-Kawai layout algorithm is shown in Algorithm 2.

ALGORITHM 1: Spring Layout Algorithm

```

1: Input: Graph  $G = (V, E)$ 
2: Output: Node positions
3: Initialize positions of all nodes randomly in a 2D space
4: Set parameters:
5:   Area (size of the canvas)
6:    $k \leftarrow \sqrt{\text{area}/|V|}$  (optimal pairwise distance)
7:   Temperature  $T$  (controls maximum displacement)
8: repeat
9:   Calculate repulsive forces:
10:  for each pair of nodes  $(u, v) \in V$  do
11:     $F_{\text{rep}}(u, v) \leftarrow -C_{\text{rep}} \cdot \frac{k^2}{\text{distance}(u, v)}$ 
12:  end for
13:  Calculate attractive forces:
14:  for each edge  $(u, v) \in E$  do
15:     $F_{\text{att}}(u, v) \leftarrow -C_{\text{att}} \cdot \frac{\text{distance}(u, v)^2}{k}$ 
16:  end for
17:  Update node positions:
18:  for each node  $u \in V$  do
19:    Compute net force  $F(u) = \sum F_{\text{rep}} + \sum F_{\text{att}}$ 
20:    Update position:
21:     $\text{pos}(u) \leftarrow \text{pos}(u) + \min(T, \|F(u)\|) \cdot \text{dir}(F(u))$ 
22:  end for
23:  Reduce temperature:  $T \leftarrow T \cdot \text{cooling\_factor}$ 
24: until System stabilizes or reaches maximum iterations
25: return Final positions of nodes

```

B Detailed Experiment Information

In this section, we provide detailed descriptions of experimental tasks, comprehensive settings for the experiments, and additional experimental results and analysis.

B.1 Experimental Tasks

To comprehensively evaluate our framework, we consider four well-known graph property detection tasks: Hamiltonian cycle problem [18], planarity verification [25], claw-free graph classification [14], and tree recognition [40]. The details of them are described below.

1) **Hamiltonian cycle problem:** This task determines whether a graph contains a cycle that visits each node exactly once before returning to the starting node. As an NP-hard problem, it is computationally challenging and widely used as a benchmark for combinatorial optimization over graphs, with applications in various areas such as routing, scheduling, and bioinformatics.

2) **Planarity verification:** This task determines whether a graph can be embedded in the plane without edge crossings. Planarity

ALGORITHM 2: Kamada-Kawai Layout Algorithm

```

1: Input: Graph  $G = (V, E)$ , shortest path distances  $d_{ij}$  for all pairs of nodes
2: Output: Node positions
3: Initialize positions of all nodes randomly in a 2D space
4: Define the energy function:
5:    $E = \sum_{(i,j)} k_{ij} \cdot (\|\text{pos}(i) - \text{pos}(j)\| - d_{ij})^2$ ,
6:   where  $k_{ij} = \text{constant}/d_{ij}^2$ 
7: repeat
8:   for each node  $u \in V$  do
9:     Compute partial derivatives of  $E$ :
10:     $\frac{\partial E}{\partial x_u} = \sum_{j \neq u} k_{uj} \cdot (\text{pos}_x(u) - \text{pos}_x(j)) \cdot (\|\text{pos}(u) - \text{pos}(j)\| - d_{uj}) / \|\text{pos}(u) - \text{pos}(j)\|$ 
11:     $\frac{\partial E}{\partial y_u} = \sum_{j \neq u} k_{uj} \cdot (\text{pos}_y(u) - \text{pos}_y(j)) \cdot (\|\text{pos}(u) - \text{pos}(j)\| - d_{uj}) / \|\text{pos}(u) - \text{pos}(j)\|$ 
12:    Update positions using gradient descent:
13:     $\text{pos}_x(u) \leftarrow \text{pos}_x(u) - \text{learning\_rate} \cdot \frac{\partial E}{\partial x_u}$ 
14:     $\text{pos}_y(u) \leftarrow \text{pos}_y(u) - \text{learning\_rate} \cdot \frac{\partial E}{\partial y_u}$ 
15:   end for
16:   Check convergence: Stop if changes in positions are below a threshold
17: until Energy  $E$  converges or reaches maximum iterations
18: return Final positions of nodes

```

verification is essential in applications like circuit design, geographical mapping, and network visualization, as planar graphs reduce visual complexity and enhance interpretability.

3) **Claw-free graph classification:** A graph is Claw-free if it does not contain a star with three leaves, $K_{1,3}$, as an induced subgraph. Claw-free graphs exhibit unique structural properties that are useful in studying network stability and chemical bonding.

4) **Tree recognition:** A tree is a connected and acyclic graph with n vertices and $n - 1$ edges, commonly used to represent hierarchical data and network structures. Tree structure recognition is crucial in fields such as spanning tree algorithms, taxonomy generation, and decision-making processes.

B.2 Synthetic Dataset

This section describes the procedures used to generate synthetic graph datasets at two scales: *large* graphs with 401-500 nodes, and *huge* graphs with 901-1000 nodes. All graphs are undirected, unweighted, and stored as adjacency matrices.

B.2.1 Hamiltonian vs. Non-Hamiltonian Graphs.

Hamiltonian graphs: Each graph is initialized as a simple cycle on n nodes, guaranteeing the existence of at least one Hamiltonian cycle. To introduce structural diversity, extra edges are randomly added between non-consecutive node pairs (i, j) with a fixed probability $p_{\text{extra}} = 0.005$, where $j \neq (i+1) \bmod n$ and $(i, j) \neq (0, n-1)$.

Non-Hamiltonian graphs: Starting from the same n -node cycle, we randomly remove a fixed number of cycle edges, between 40 and 80 for large graphs, and between 200 and 250 for huge graphs, to break potential Hamiltonian cycles. Subsequently, extra edges are added using the same probability $p_{\text{extra}} = 0.005$, excluding the previously removed and wraparound edges.

B.2.2 Planar vs. Non-Planar Graphs.

Planar graphs: Each graph begins as a uniformly random spanning tree with n nodes, ensuring connectivity and planarity. Additional edges are then incrementally added by randomly sampling node pairs and accepting only those that preserve planarity, as verified using the Boyer-Myrvold planarity test [25]. This process continues until the graph reaches approximately $\lceil 1.6n \rceil$ edges, resulting in near-sparse planar graphs.

Non-planar graphs: Starting from a sparse planar graph, we randomly embed a forbidden subgraph, either K_5 or $K_{3,3}$, by adding the edges required to form it among randomly selected nodes. To maintain the total edge count, an equal number of unrelated edges are removed. This guarantees non-planarity by introducing a subgraph that violates Kuratowski's theorem.

B.2.3 Claw-Free vs. Non-Claw-Free Graphs.

Claw-free graphs: These graphs are constructed as line graphs of random trees on $n + 1$ nodes, which are guaranteed to be claw-free by definition. To reach the target edge density ($\rho = 0.008$ for large graphs and $\rho = 0.005$ for huge graphs), we uniformly add edges from the set of non-edges, while ensuring that the total edge count does not exceed the maximum possible for the given n .

Non-claw-free graphs: These graphs are derived from the claw-free graphs by explicitly planting $K_{1,3}$ (claw) subgraphs. We insert a fixed number of claws (15 per large graph and 50 per huge graph) by selecting disjoint node sets and adding three edges per claw. To maintain the original edge count, an equal number of unrelated edges are randomly removed.

B.2.4 Tree vs. Non-Tree Graphs.

Tree graphs: Each instance is produced as a uniformly sampled spanning tree on n nodes, containing exactly $n - 1$ edges.

Non-tree graphs: Each graph is constructed by modifying a random spanning tree through either adding or removing edges. For edge addition, extra edges are sampled uniformly from the set of non-edges, with the number of added edges selected uniformly between $\lceil 0.05(n - 1) \rceil$ and $\lceil 0.12(n - 1) \rceil$. For edge removal, a comparable proportion of edges is randomly deleted. This process ensures that the resulting graph either contains cycles or becomes disconnected, and is therefore not a tree.

B.3 Training Settings

In this section, we provide a detailed description of training settings for VSAL and other learning baselines. In particular, all experiments are conducted using 4 NVIDIA L40S GPUs.

- **VSAL:** A detailed summary of all hyperparameter settings in VSAL can be found in Table 5. Moreover, the empirical analysis for λ_c , λ_{gp} , m , r and δ are provided in Appendix B.4.
- **VN-Solver:** The VN-Solver is trained using the Adam optimizer with a learning rate of $1e-3$ and employs two fixed layouts, including circular and spiral layouts.
- **Graphormer** [49] and **Graphormer-GD** [50]: They are trained using Adam optimizer with a learning rate of $1e-5$.
- **EquiformerV2:** We follow the settings of [32] and train it using the AdamW optimizer with a learning rate of $1e-4$.
- **GraphsGPT:** We follow the settings of [16], and it is trained using the AdamW optimizer with a learning rate of $1e-4$.

Table 5: Hyperparameter settings in VSAL.

Hyperparameter	Value
Learning rate of generator	0.0001
Learning rate of discriminator	0.0005
Learning rate of classifier	0.00001
Classification loss weight λ_c	30
Gradient penalty coefficient λ_{gp}	5
The number of sampled noise vectors m	10
Hidden dimension of generator d_g	128
Hidden dimension of discriminator d_s	128
Dimension of latent noise space d_z	128
Visualization height and width $H \times W$	224, 380, 528
Control the degree of node influence r	2
Control the degree of edge influence δ	1
Control the sharpness of approximation β	10
The radius of the Gaussian kernel K	5
The strength of the Gaussian smoothing σ	2
The radius of the circular initial layout r_1	1
The offset factor of the spiral initial layout r_2	0.2
The number of shells in the shell initial layout S	2
The radius of the first shell $r_{j,1}$	0.5
The radius of the second shell $r_{j,2}$	1
The bound of the uniform initial layout b	1

Table 6: Hyperparameter analysis for λ_c , λ_{gp} , m , r and δ .

λ_c	F1	λ_{gp}	F1	m	F1	r	F1	δ	F1
1	0.96 _(0.01)	1	0.97 _(0.01)	5	0.96 _(0.00)	1	0.95 _(0.01)	0.5	0.96 _(0.01)
10	0.97 _(0.01)	5	0.98 _(0.00)	8	0.97 _(0.00)	2	0.98 _(0.00)	1	0.98 _(0.00)
30	0.98 _(0.00)	10	0.97 _(0.00)	10	0.98 _(0.00)	5	0.91 _(0.01)	2	0.94 _(0.01)
50	0.95 _(0.01)	20	0.96 _(0.01)	15	0.97 _(0.00)	10	0.86 _(0.01)	5	0.88 _(0.01)

B.4 Analysis of Key Hyperparameters

We analyze the impact of key hyperparameters by conducting experiments using DenseGCN-based VSAL-V on the Hamiltonian cycle problem with 1000 medium training graphs. The evaluated hyperparameters include the classification loss weight λ_c , gradient penalty coefficient λ_{gp} , the number of sampled noise vectors m , the node influence factor r , and the edge influence factor δ . The results are summarized in Table 6. Based on these results and grid search, we set $\lambda_c = 30$, $\lambda_{gp} = 5$, $m = 10$, $r = 2$ and $\delta = 1$ for all experiments.

B.5 Additional results and Analysis

VSAL is robust to different initial layouts. Table 7 compares the performance of VSAL with DenseGCN against VN-Solver on four tasks using different initial layouts. VSAL consistently achieves high performance regardless of the initial layouts, while the results of VN-Solver vary significantly. This confirms the effectiveness and robustness of adaptive layouts again.

Inference time comparison. To evaluate the efficiency of VSAL, we compare its inference time with several traditional exact algorithms used for graph property detection:

Table 7: F1 scores for different initial layouts across four tasks, using a training size of 1000 (and 200 for the small tree dataset).

	Hamiltonian cycle problem								Claw-free classification					
	VSAL-V-224 (DenseGCN)				VN-Solver				VSAL-V-224 (DenseGCN)			VN-Solver		
	Small	Medium	Large	Huge	Small	Medium	Large	Huge	Small	Large	Huge	Small	Large	Huge
Circular	0.90 _(0.01)	0.98 _(0.00)	0.91 _(0.00)	0.90 _(0.01)	0.83 _(0.03)	0.95 _(0.01)	0.57 _(0.03)	0.53 _(0.01)	0.93 _(0.01)	0.93 _(0.01)	0.92 _(0.01)	0.86 _(0.01)	0.60 _(0.01)	0.58 _(0.02)
Spiral	0.90 _(0.01)	0.98 _(0.01)	0.91 _(0.00)	0.90 _(0.01)	0.80 _(0.02)	0.95 _(0.01)	0.81 _(0.01)	0.76 _(0.01)	0.93 _(0.01)	0.93 _(0.01)	0.91 _(0.01)	0.85 _(0.01)	0.68 _(0.02)	0.62 _(0.02)
Shell	0.89 _(0.01)	0.97 _(0.00)	0.91 _(0.01)	0.89 _(0.00)	0.74 _(0.05)	0.94 _(0.02)	0.61 _(0.02)	0.59 _(0.01)	0.93 _(0.01)	0.93 _(0.01)	0.90 _(0.01)	0.86 _(0.02)	0.61 _(0.01)	0.58 _(0.01)
Uniform	0.87 _(0.01)	0.96 _(0.01)	0.89 _(0.01)	0.87 _(0.00)	0.58 _(0.01)	0.65 _(0.00)	0.53 _(0.02)	0.51 _(0.02)	0.90 _(0.01)	0.89 _(0.01)	0.87 _(0.01)	0.73 _(0.01)	0.56 _(0.02)	0.52 _(0.01)
	Planarity verification								Tree recognition					
	VSAL-V-224 (DenseGCN)				VN-Solver				VSAL-V-224 (DenseGCN)			VN-Solver		
	Small	Medium	Large	Huge	Small	Medium	Large	Huge	Small	Large	Huge	Small	Large	Huge
Circular	0.89 _(0.01)	0.97 _(0.00)	0.92 _(0.01)	0.91 _(0.01)	0.85 _(0.02)	0.95 _(0.01)	0.51 _(0.02)	0.51 _(0.01)	0.96 _(0.01)	0.94 _(0.01)	0.92 _(0.00)	0.83 _(0.02)	0.76 _(0.01)	0.64 _(0.02)
Spiral	0.89 _(0.01)	0.96 _(0.00)	0.92 _(0.01)	0.91 _(0.01)	0.84 _(0.01)	0.94 _(0.02)	0.53 _(0.01)	0.51 _(0.02)	0.95 _(0.01)	0.94 _(0.01)	0.92 _(0.01)	0.81 _(0.03)	0.77 _(0.02)	0.68 _(0.02)
Shell	0.89 _(0.01)	0.97 _(0.01)	0.91 _(0.01)	0.89 _(0.01)	0.84 _(0.01)	0.96 _(0.01)	0.53 _(0.01)	0.51 _(0.01)	0.96 _(0.01)	0.93 _(0.01)	0.91 _(0.01)	0.76 _(0.04)	0.71 _(0.03)	0.63 _(0.02)
Uniform	0.86 _(0.02)	0.93 _(0.01)	0.88 _(0.01)	0.85 _(0.01)	0.74 _(0.02)	0.85 _(0.01)	0.51 _(0.01)	0.47 _(0.02)	0.92 _(0.01)	0.90 _(0.01)	0.87 _(0.01)	0.72 _(0.03)	0.66 _(0.02)	0.55 _(0.02)

Table 8: Average running time on one graph (small and huge graph) for different tasks (in seconds).

Task	Method	Small	Huge
Ham	Held-Karp algorithm	1.376	> 3600
	VSAL-V-224	0.008	0.015
Planar	Hopcroft-Tarjan algorithm	0.00003	0.0014
	VSAL-V-224	0.007	0.012
Claw	Brute-force induced $K_{1,3}$ check	0.016	1361.258
	VSAL-V-224	0.007	0.013
Tree	DFS on Tree	0.000004	0.0002
	VSAL-V-224	0.005	0.009

Table 9: Average GPU memory usage during the training on the Hamiltonian cycle problem with a batch size of 1.

Graph Size	VSAL-V-224	Graphormer
Small	1261.8 MB	945.8 MB
Medium	1261.8 MB	1057.8 MB
Large	2719.8 MB	17521.8 MB
Huge	2965.8 MB	39785.8 MB

Table 10: Ablation results (F1 scores) of Graphormer-based VSAL-V. “L” and “H” refer to large and huge graphs, respectively. Exp-A: w/o Pretraining, Exp-B: w/o Refinement, Exp-C: w/o Gaussian Smoothing, Exp-D: w/o Visualization Module.

		VSAL-V	Exp-A	Exp-B	Exp-C	Exp-D
Ham	L	0.94 _(0.01)	0.90 _(0.01)	0.91 _(0.01)	0.92 _(0.01)	0.91 _(0.01)
	H	0.92 _(0.01)	0.87 _(0.01)	0.88 _(0.01)	0.89 _(0.00)	0.89 _(0.01)
Planar	L	0.94 _(0.01)	0.89 _(0.01)	0.91 _(0.01)	0.92 _(0.01)	0.91 _(0.01)
	H	0.93 _(0.01)	0.87 _(0.01)	0.89 _(0.00)	0.90 _(0.01)	0.89 _(0.01)
Claw	L	0.95 _(0.01)	0.88 _(0.01)	0.91 _(0.01)	0.93 _(0.01)	0.90 _(0.01)
	H	0.94 _(0.01)	0.86 _(0.01)	0.89 _(0.01)	0.91 _(0.01)	0.89 _(0.01)
Tree	L	0.96 _(0.01)	0.90 _(0.01)	0.92 _(0.01)	0.94 _(0.00)	0.93 _(0.01)
	H	0.94 _(0.01)	0.87 _(0.01)	0.90 _(0.01)	0.91 _(0.01)	0.90 _(0.01)

- **Hamiltonian cycle problem:** Held-Karp algorithm [23], with exponential time complexity $\Theta(2^n n^2)$.
- **Planarity verification:** Hopcroft-Tarjan algorithm [25], with linear time complexity $O(n)$.
- **Claw-free graph classification:** Brute-force induced sub-graph check for $K_{1,3}$, with time complexity $O(n^4)$.
- **Tree recognition:** Depth-First Search (DFS), with linear time complexity $O(n)$.

As shown in Table 8, VSAL significantly reduces computational overhead compared to algorithms with time complexity greater than $O(n)$, particularly on huge graphs. These results confirm that VSAL can achieve extreme efficiency without sacrificing much efficacy, highlighting again the advantages of learning-based methods for graph property detection.

GPU memory usage comparison. We compare the GPU memory usage of VSAL and Graphormer during training with a batch size of 1, and the results are presented in Table 9. VSAL demonstrates relatively stable memory consumption, ranging from 1261.8 MB on small graphs to 2965.8 MB on huge graphs. In contrast, the GPU memory usage of Graphormer increases dramatically, from 945.8 MB on small graphs to 39785.8 MB on huge graphs, indicating poor scalability. This difference stems from their data representations: the memory usage of VSAL depends on fixed image resolution (set to 224×224 here, but adjustable), whereas Graphormer scales with the size of the adjacency matrix (1000×1000 for huge graphs). Although image representations may abstract some details, their resolution can be tuned to balance efficiency and accuracy. Indeed, the graph visualizations still retain key structural features for graph property detection, thus allowing VSAL to process much larger graphs while maintaining comparable performance and significantly lower memory overhead.

Ablation study. The complementary results of ablation study on large and huge graphs are summarized in Table 10. This further confirm that all these modules are helpful in improving graph property detection performance.